Agentes IoT

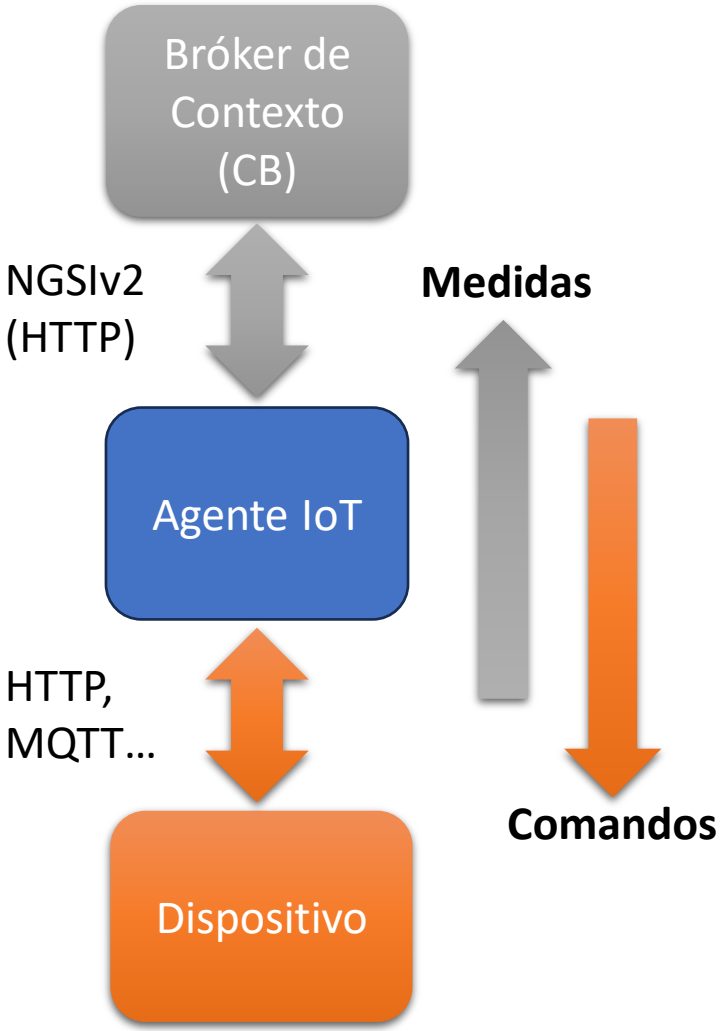
Gestión de Datos en Entornos Inteligentes (XDEI)

IoT Sensors

Agentes IoT

IoT Sensors

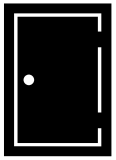
- **IoT *(Internet of Things)***
  - Formada por dispositivos electrónicos que pueden actuar como **sensores** y/o **actuadores**
  - Un **dispositivo IoT** es un objeto físico que existe en el mundo real → lo **representaremos como una entidad** única dentro del contexto
- **Agente IoT**
  - *Middleware* entre dispositivos y bróker de contexto
  - Dispositivos: protocolos HTTP, MQTT, AMQP, LWM2M…
  - Bróker de contexto: NGSIv2 (HTTP)



IoT Sensors
Ejemplos de dispositivos del tutorial

- **Campana** *(Bell)*
  - **Actuador**: hacerla sonar

- **Sensor de movimiento** *(Motion Sensor)*
  - **Sensor**: se activa y envía un conteo cuando alguien está cerca

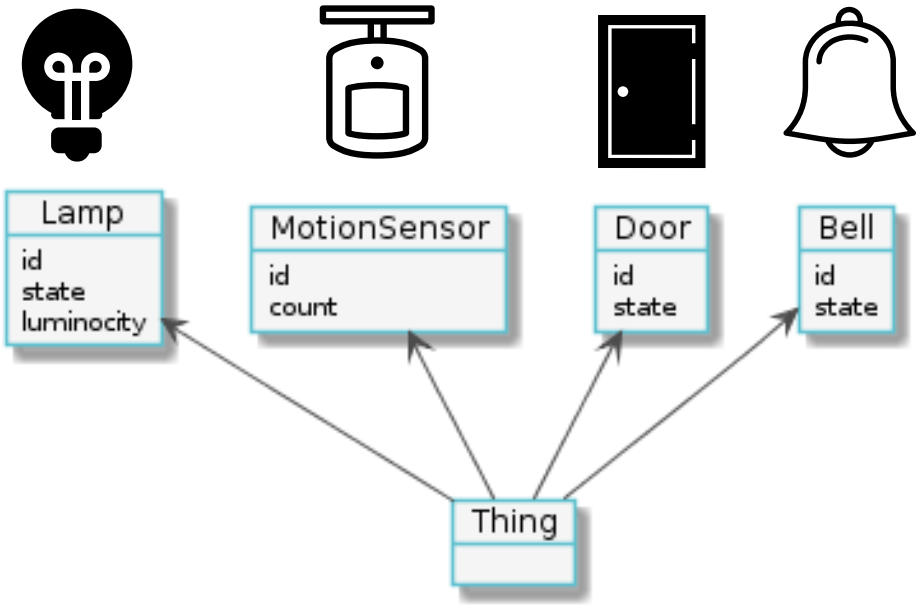


- **Puerta Inteligente** *(Smart Door)*
  - **Actuador**: bloquear *(lock)* o desbloquear *(unlock)*
  - **Sensor**: estado actual (OPEN, CLOSED, LOCKED)

- **Lámpara inteligente** *(Smart Lamp)*
  - **Actuador**: encender o apagar
  - **Sensor**: estado actual (ON, OFF). Cuando está encendida, su sensor de movimiento verifica si se necesita luz y atenuará su intensidad si no hay nadie cerca. También informa sobre la luminosidad actual de la bombilla.

IoT Sensors
Ejemplos de dispositivos del tutorial (II)

- **Modelo de datos**

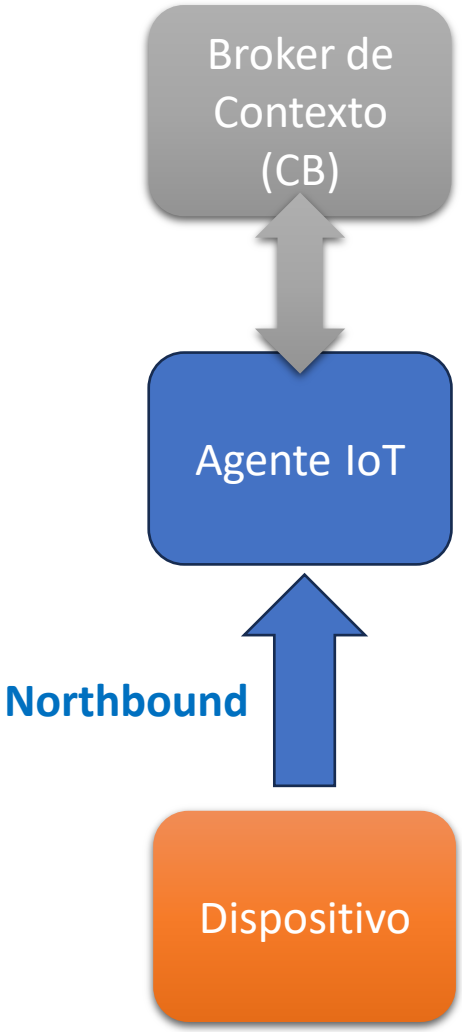


IoT Sensors
Tráfico Dispositivo → Agente IoT

- **Northbound (envío de medidas al Agente IoT)**
  - **GET** http://<iot-agent>/<resource-path>
    Ej.: <resource-path> = /iot/d
    Params.:
    - **i** *(Device ID)*. ID del dispositivo (único para una clave del API)
    - **k** *(API key)*. Clave del API del servicio en el que está registrado el dispositivo
    - **t**. *Timestamp* (opcional)
    - **d**. Carga útil UltraLight 2.0 (opcional)

  - **POST** http://<iot-agent>/<resource-path>
    Params.:
    - **i** *(Device ID)*. ID del dispositivo (único para una clave del API)
    - **k** *(API key)*. Clave del API del servicio en el que está registrado el dispositivo
    - **t**. *Timestamp* (opcional)

    Body:
    - Carga útil UltraLight 2.0 (opcional)

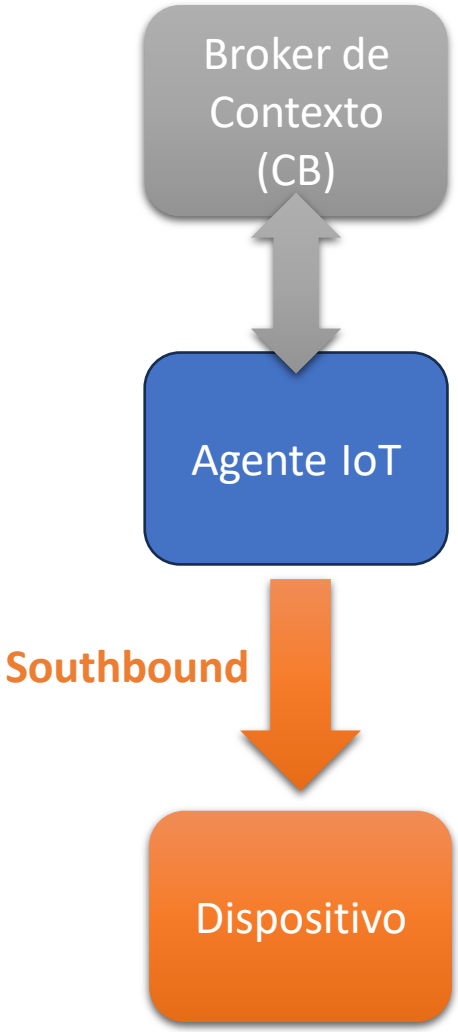


IoT Sensors
Tráfico Agente IoT → Dispositivo

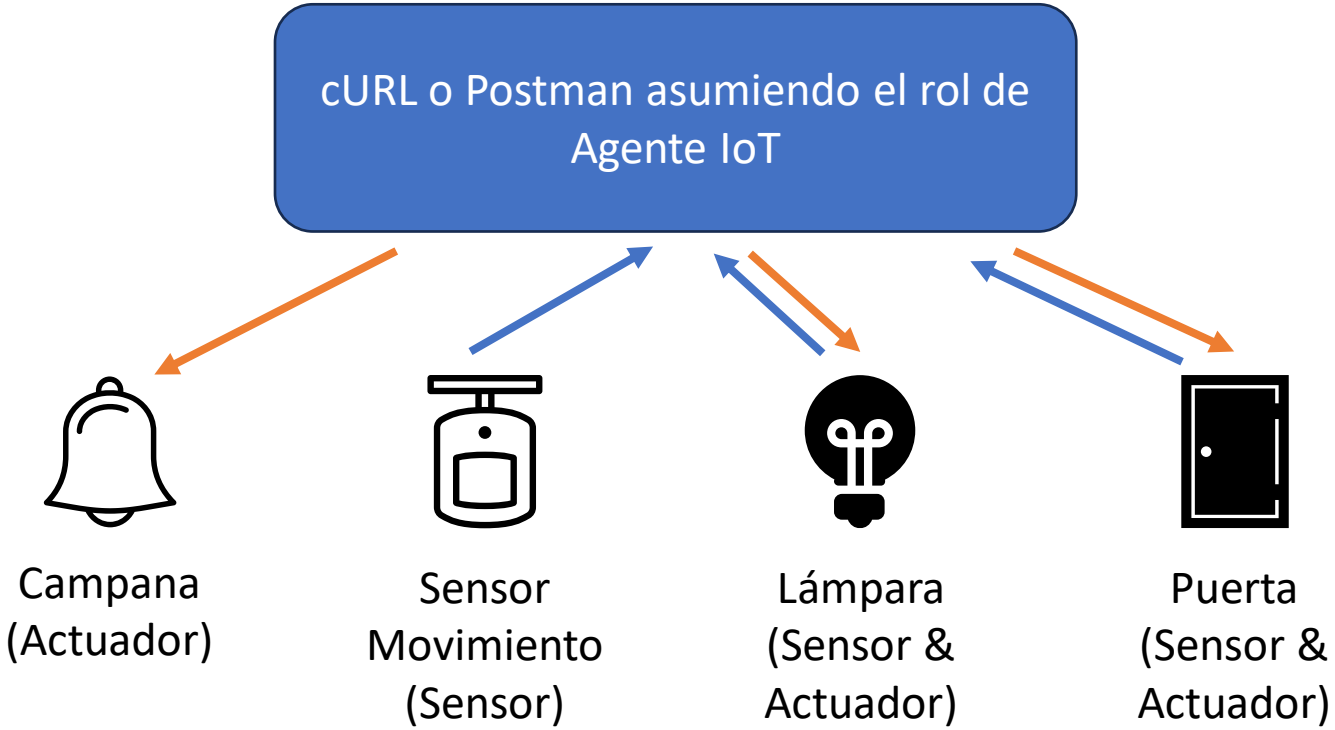
- **Southbound (envío de comandos al dispositivo)**
  - **POST** http://<device-url>
    Body: Carga útil UltraLight 2.0

    `<device-id>@<command>|<param|<param>`

  - Ej.: `urn:ngsi-ld:Robot:001@turn|left|30`



IoT Sensors
Arquitectura del tutorial



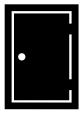
- **Comandos**: sentido Southbound al **dispositivo** (escuchando en puerto **3001**)
- **Medidas**: sentido Northbound al **Agente IoT** (escuchando en puerto **7896**)

IoT Sensors
Ejemplos de medidas y comandos

- **Desbloquear puerta**
  - **POST** http://localhost:3001/iot/door001
    Body: urn:ngsi-ld:Door:001@unlock
    - *Resp.*: urn:ngsi-ld:Door:001@unlock| unlock OK
  - Una vez desbloqueada, la puerta **se abrirá y cerrará cuando entre un cliente**
  - El **detector de movimiento notificará** acerca del paso de los clientes **enviando una medida** al Agente IoT
  - Cada **cambio de estado** (LOCKED, OPEN, CLOSED) de la puerta **se envía como una medida** al Agente IoT

- **Abrir puerta**
  - **POST** http://localhost:3001/iot/door001
    Body: urn:ngsi-ld:Door:001@open
    - *Resp.*: urn:ngsi-ld:Door:001@open| open OK

- **Cerrar puerta**
  - **POST** http://localhost:3001/iot/door001
    Body: urn:ngsi-ld:Door:001@close
    - *Resp.*: urn:ngsi-ld:Door:001@close| close OK



- **Bloquear puerta**
  - **POST** http://localhost:3001/iot/door001
    Body: urn:ngsi-ld:Door:001@lock
    - *Resp.*: urn:ngsi-ld:Door:001@lock| lock OK

IoT Sensors
Ejemplos de medidas y comandos (II)

- **Tocar campana**
  - **POST** http://localhost:3001/iot/bell001
    Body: urn:ngsi-ld:Bell:001@on
    - *Resp.*: urn:ngsi-ld:Bell:001@ring| ring OK

- **Encender lámpara**
  - **POST** http://localhost:3001/iot/lamp001
    Body: urn:ngsi-ld:Lamp:001@on
    - *Resp.*: urn:ngsi-ld:Lamp:001@on| on OK

  - Una vez encendida, el nivel de **luminosidad de la lámpara irá variando** de acuerdo a las detecciones del **sensor de movimiento**.
  - **Cada cambio** en el nivel de luminosidad **se envía como medida** al Agente IoT
- **Apagar lámpara**
  - **POST** http://localhost:3001/iot/lamp001
    Body: urn:ngsi-ld:Lamp:001@off
    - *Resp.*: urn:ngsi-ld:Lamp:001@off| off OK

  - Una vez apagada, no se producen cambios en su luminosidad
  - El último valor *(s|OFF|l|0)* **se envía como medida** al Agente IoT

- Podemos **ver las medidas enviadas al Agente IoT en la sección Northbound Traffic del Device Monitor** del tutorial

Agente IoT (UltraLight)

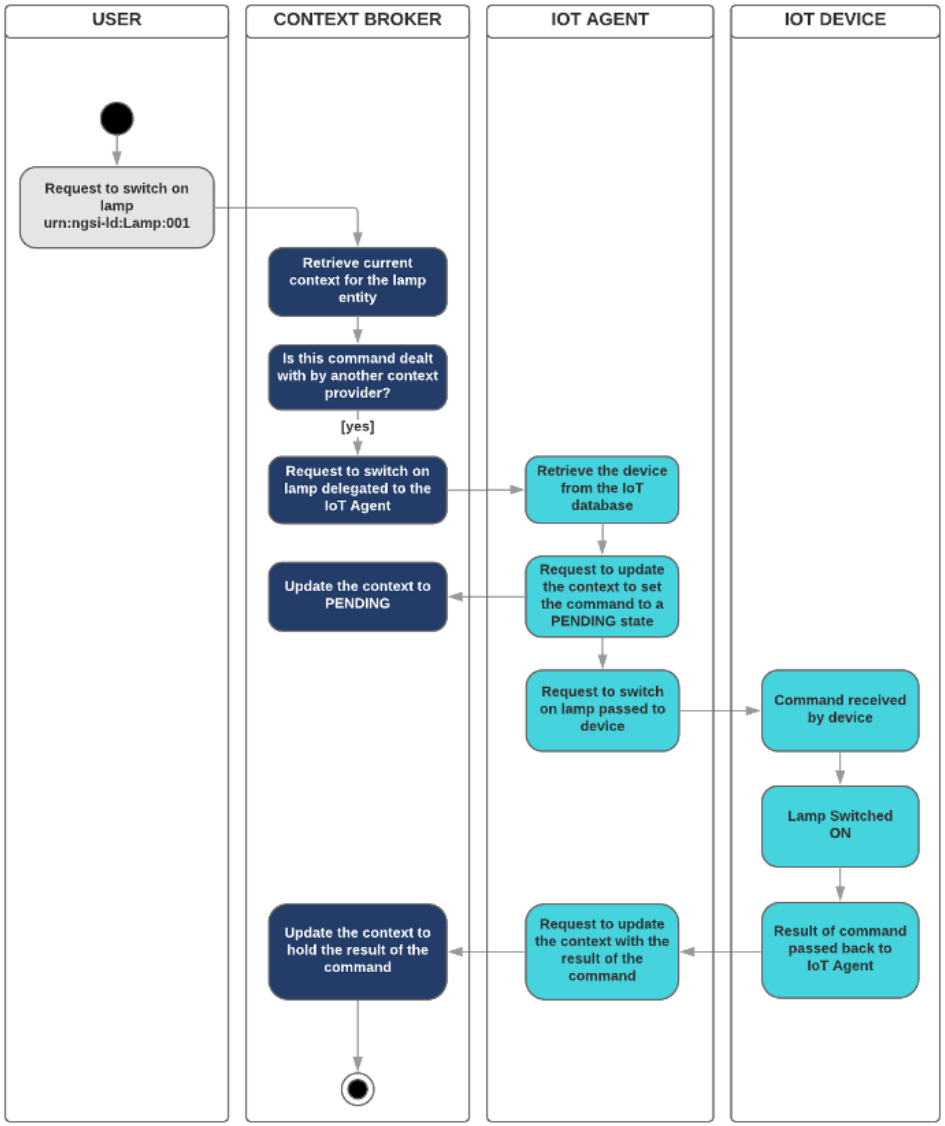
Agentes IoT

Agente IoT UltraLight
Tipos de Agentes IoT

- **Agentes IoT** resultantes de combinar
  - **Carga útil**
    - UltraLight 2.0
    - JSON
  - **Transporte**
    - HTTP
    - MQTT
    - MQTT sobre IOTA Tangle
- **Otros agentes IoT**
  - LWM2M (sobre CoAP)
  - LoRaWAN
  - OPC-UA

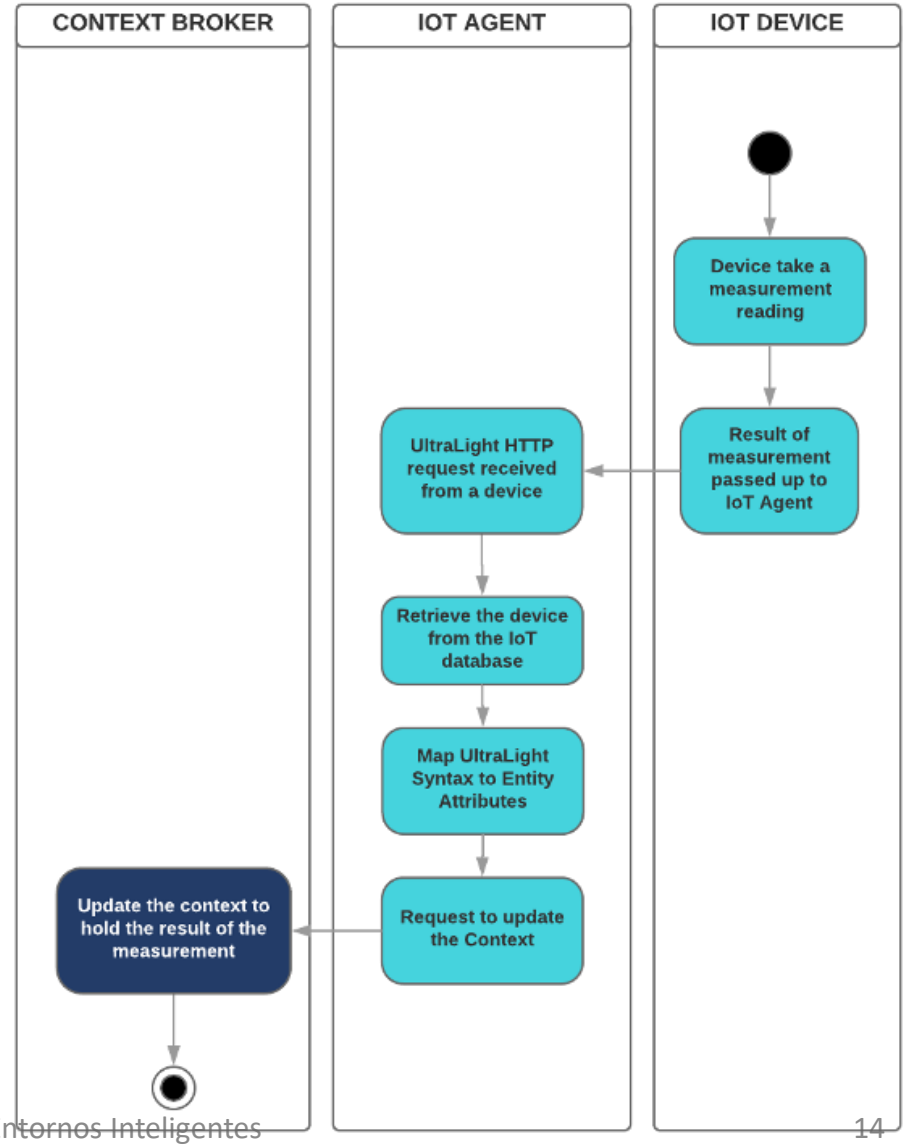
Agente IoT UL
Tráfico *Southbound* (Comandos)

- **Protocolos**
  - Usuario – CB: NGSI
  - CB – Agente IoT: NGSI
  - Agente IoT – Dispositivo: Nativo



Agente IoT UL
Tráfico *Northbound* (Medidas)

- Protocolos
  - CB – Agente IoT: NGSI
  - Agente IoT – Dispositivo: Nativo

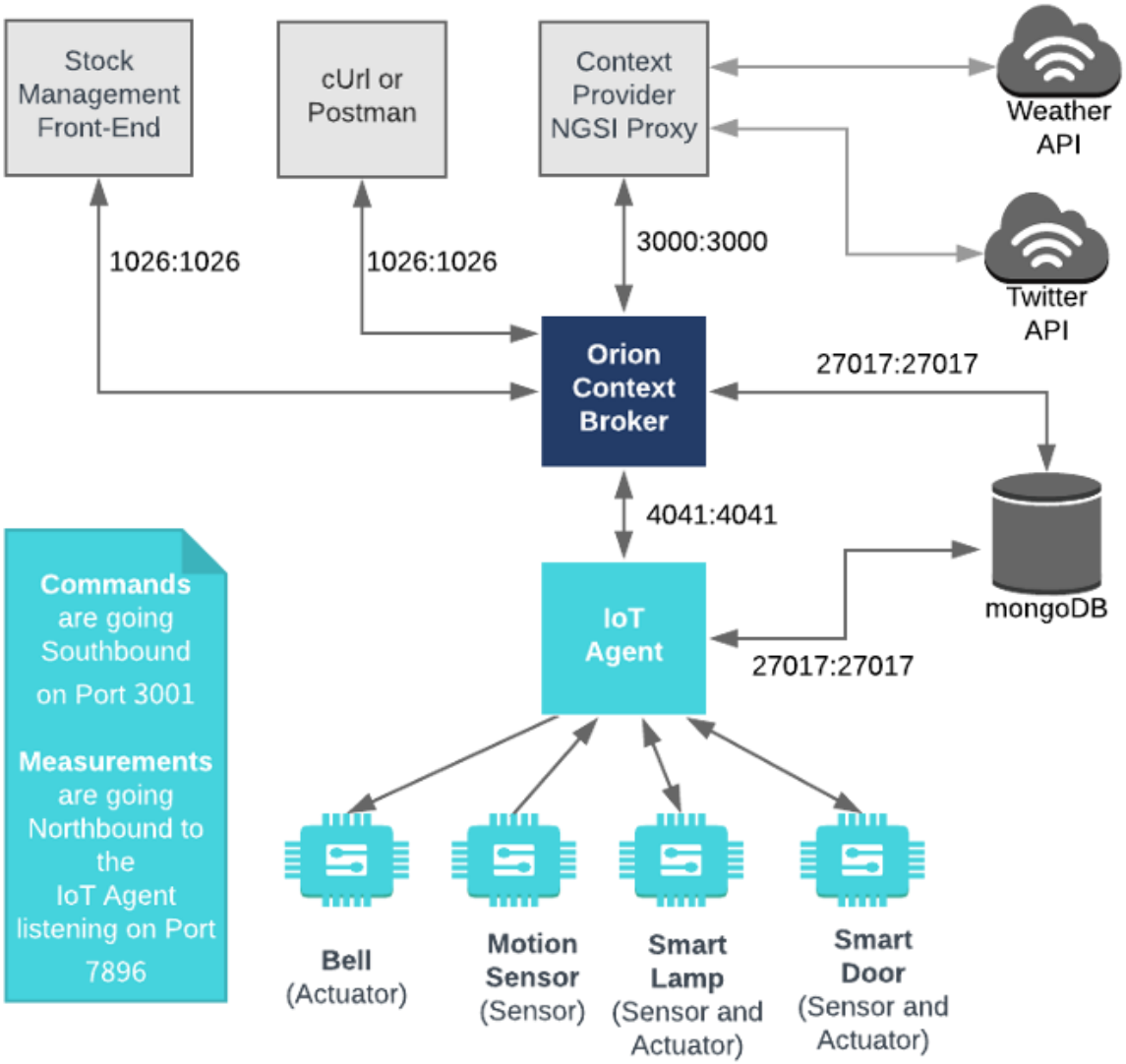


Agente IoT UL
Funcionalidades comunes

- Ofrecer una **ubicación estándar** para escuchar **actualizaciones** de
  - **Dispositivos**
  - **Datos contextuales**
- Mantener una **lista de dispositivos** y **asignar atributos de datos contextuales a la sintaxis del dispositivo**
- **Autorización** de **seguridad**

- Estas funcionalidades comunes a **cualquier tipo de agente IoT** están implementadas en la IoT Agent Framework Library

Agentes IoT
Arquitectura del tutorial



Agente IoT UL
Conexión de dispositivos al Agente IoT

- Dispositivos tienen un **atributo *device_id***
  - Puede haber **varios dispositivos** con el **mismo *device_id***
- Peticiones de aprovisionamiento de dispositivos requieren
  - ***fiware-service***. Las entidades de un servicio se almacenan en una **base de datos MongoDB separada**
  - ***fiware-servicepath***. Permite distinguir **grupos de dispositivos** dentro de un **mismo *fiware-service***
- Un ***device_id*** debe ser **único** dentro de un ***fiware_service* y *fiware-servicepath***
- Ej.: aplicación *smart city*
  - Un *fiware-service* para cada departamento (parques, transporte, recogida de residuos…)
  - Un *fiware-servicepath* para cada sección dentro de un departamento (Parque 1, Parque 2, Línea Bus 1, Línea Bus 2, Papelera 1, Papelera 2…)

Agente IoT UL
Aprovisionamiento grupo de dispositivos

- Dentro de un *fiware-service*, *fiware-servicepath* podemos tener varios grupos de dispositivos *(service groups)*
- Un grupo de dispositivos suele agrupar dispositivos de un mismo tipo, p. ej. detectores de movimiento
- **POST** http://<iot-agent>**:4041/iot/services**
  - *Headers*: *fiware-service*, *fiware-servicepath*
  - *Body*: *services*, lista de grupos de dispositivos donde cada grupo tiene atributos *apikey*, *cbroker*, *entity_type*, *resource*
- Ej.:

```
curl -X POST 'http://localhost:4041/iot/services' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' \
  -H 'fiware-servicepath: /' \
  -d '{ "services": [ {
                        "apikey":      "4jggokgpepnvsb2uv4s40d59ov",
                        "cbroker":     "http://orion:1026",
                        "entity_type": "Thing",
                        "resource":    "/iot/d"
                       } ] }'
```

Agente IoT UL
Aprovisionamiento grupo de dispositivos

- El agente IoT acepta cualquier medida enviada por un dispositivo siempre que sean correctos:
  - *fiware-service, fiware-servicepath*
  - *apikey*
  - *resource-path*

  Identifican unívocamente el grupo de dispositivos
- Si el dispositivo NO ha sido aprovisionado previamente, el agente IoT asume que el tipo de entidad que le corresponde es la indicada como *entity_type* al crear el grupo de dispositivos (*Thing* en el ejemplo anterior)
- Asumiendo ese tipo de entidad, traslada las medidas a Orion igual que haría con un dispositivo aprovisionado previamente

Agente IoT UL
Grupo de dispositivos – Operac. CRUD

- Mediante **POST** / **GET** / **PUT** / **DELETE** en http://<iot-agent>**:4041/iot/services**
- Ej.: Lectura de los grupos de dispositivos para un *resource path*

```
curl -G -X GET 'http://localhost:4041/iot/services' \
  -d 'resource=/iot/d' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Ej.: Listado de todos los grupos de dispositivos

```
curl -X GET 'http://localhost:4041/iot/services' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Ej.: Actualizar un grupo de dispositivos

```
curl -iX PUT http://localhost:4041/iot/services?resource=/iot/d&apikey=4j…ov' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "entity_type": "IoT-Device" }'
```

- Ej.: Borrar un grupo de dispositivos

```
curl -iX DELETE \
  'http://localhost:4041/iot/services/?resource=/iot/d&apikey=4j…ov' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

Agente IoT UL
Aprovisionamiento sensor

- **POST** http://<iot-agent>**:4041/iot/devices**
  - *Headers*: *fiware-service*, *fiware-servicepath*
  - *Body*: *devices*, lista de dispositivos donde cada dispositivo tiene atributos ***device_id***, ***entity_name***, ***entity_type***, *timezone*, ***attributes***, *static_attributes*
- Ej.:

```
curl -X POST 'http://localhost:4041/iot/devices' \
     -H 'Content-Type: application/json' \
     -H 'fiware-service: openiot' \
     -H 'fiware-servicepath: /' \
     -d '{ "devices": [ {
        "device_id":   "motion001",
        "entity_name": "urn:ngsi-ld:Motion:001",
        "entity_type": "Motion",
        "timezone":    "Europe/Berlin",
        "attributes": [{"object_id": "c", "name": "count",
                        "type": "Integer" }],
        "static_attributes": [
             { "name":"refStore", "type": "Relationship",
               "value": "urn:ngsi-ld:Store:001" } ] } ] }'
```

Asociación dispositivo – entidad

Agente IoT UL
Procesado de medida en el agente IoT

- **Ej.: simulamos que el sensor de movimiento envía medida al agente IoT**

```
curl -iX POST \
  'http://localhost:7896/iot/d?k=4jggokgpepnvsb2uv4s40d59ov&i=motion001' \
  -H 'Content-Type: text/plain' \
  -d 'c|1'
```

- Como el **dispositivo** ya está **aprovisionado**, el **agente IoT**
  1. Realiza una **correspondencia** entre los **atributos del dispositivo y de la entidad que lo representa** en el contexto
  2. **Actualiza** dichos atributos **en el CB**
- Si ahora leemos dicha entidad en el CB

```
curl -G -X GET \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Motion:001' \
  -d 'type=Motion' \
  -H 'fiware-service: openiot' \
  -H 'fiware-servicepath: /'
```

Agente IoT UL
Procesado de medida en el agente IoT (II)

- Obtenemos


```
{ "id": "urn:ngsi-ld:Motion:001",
  "type": "Motion",
  "TimeInstant": {
    "type": "ISO8601",
    "value": "2018-05-25T10:51:32.00Z",
    "metadata": {} },
  "count": { "type": "Integer",
             "value": "1",
             "metadata": {
               "TimeInstant": {
                 "type": "ISO8601",
                 "value": "2018-05-25T10:51:32.646Z" } } },
  "refStore": { "type": "Relationship",
                "value": "urn:ngsi-ld:Store:001",
                "metadata": {
                  "TimeInstant": {
                    "type": "ISO8601",
                    "value": "2018-05-25T10:51:32.646Z" } } }
}
```

Agente IoT UL Aprovisionamiento actuador

- **POST** http://<iot-agent>**:4041/iot/devices**
  - *Headers*: *fiware-service*, *fiware-servicepath*
  - *Body*: *devices*, lista de dispositivos donde cada dispositivo tiene atributos *device_id*, *entity_name*, *entity_type*, ***protocol***, ***transport***, ***endpoint***, ***commands***, *static_attributes*
- Ej.:

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices": [ {
            "device_id": "bell001",
            "entity_name": "urn:ngsi-ld:Bell:001", "entity_type": "Bell",
            "protocol": "PDI-IoTA-UltraLight",
            "transport": "HTTP",
            "endpoint": "http://iot-sensors:3001/iot/bell001",
            "commands": [ { "name": "ring", "type": "command" } ],
            "static_attributes": [
                {"name":"refStore", "type": "Relationship",
                 "value": "urn:ngsi-ld:Store:001"} ] } ] }'
```

Agente IoT UL
Aprovis. actuador: atributo bidireccional

- Ej.:
```
curl -L -X POST 'http://localhost:4041/iot/devices' \
      -H 'Content-Type: application/json' \
      -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
      -d '{ "devices": [ {
              "device_id": "bell002",
              "entity_name": "urn:ngsi-ld:Bell:002", "entity_type": "Bell",
              "protocol": "PDI-IoTA-UltraLight",
              "transport": "HTTP",
              "endpoint": "http://iot-sensors:3001/iot/bell002",
              "attributes": [ {
                 "name": "ring", "type": "Text", "expression": "ring",
                 "reverse": [ { "object_id": "ring",
                                "type": "Text",
                                "expression": "ring | toString()" } ] } ],
              "static_attributes": [ {
                 "name":"refStore", "type": "Relationship",
                 "value": "urn:ngsi-ld:Store:002"} ] } ] } '
```

- Este método utiliza una **suscripción en lugar de un registro**

Agente IoT UL
Aprovis. actuador: atributo bidirecc. (II)

- Si ahora **simulamos el papel del CB** y **cambiamos el valor del atributo** usando el ***Northport*** **NGSIv2 del agente IoT**

```
curl -iX POST http://localhost:4041/v2/op/update \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d ‘{ "actionType": "update",
        "entities": [ {
            "type": "Bell", "id": "urn:ngsi-ld:Bell:001",
            "ring" : { "type": "command", "value": "" } } ] }'
```

- Consultamos resultado de la ejecución del comando en el CB

```
curl -G -X GET 'http://localhost:1026/v2/entities/urn:ngsi-ld:Bell:001' \
  -d 'type=Bell' -d 'options=keyValues' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

  - Resp.:

```
{ "id": "urn:ngsi-ld:Bell:001", "type": "Bell",
  "TimeInstant": "2018-05-25T20:06:28.00Z",
  "refStore": "urn:ngsi-ld:Store:001",
  "ring_info": "OK",
  "ring_status": "OK", "ring": "" }
```

Agente IoT UL
Aprovisionamiento sensor & actuador

- Ej.: Puerta

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices": [
         {    "device_id": "door001",
              "entity_name": "urn:ngsi-ld:Door:001",
              "entity_type": "Door",
              "protocol": "PDI-IoTA-UltraLight",
              "transport": "HTTP",
              "endpoint": "http://iot-sensors:3001/iot/door001",
              "commands": [ {"name": "unlock","type": "command"},
                            {"name": "open","type": "command"},
                            {"name": "close","type": "command"},
                            {"name": "lock","type": "command"}],
              "attributes": [ {"object_id": "s",
                               "name": "state", "type":"Text"} ],
              "static_attributes": [ {"name":"refStore",
                                      "type": "Relationship",
                                      "value": "urn:ngsi-ld:Store:001"}]  } ] }'
```

Agente IoT UL
Aprovisionamiento sensor & actuador

- Ej.: Lámpara

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices":
     [
      { "device_id": "lamp001",
        "entity_name": "urn:ngsi-ld:Lamp:001",
        "entity_type": "Lamp",
        "protocol": "PDI-IoTA-UltraLight",
        "transport": "HTTP",
        "endpoint": "http://iot-sensors:3001/iot/lamp001",
        "commands": [ {"name": "on","type": "command"},
                      {"name": "off","type": "command"}],
        "attributes": [{"object_id": "s", "name": "state", "type":"Text"},
                       {"object_id": "l", "name": "luminosity", "type":"Integer"}],
       "static_attributes": [ { "name":"refStore",
                                "type": "Relationship",
                                "value": "urn:ngsi-ld:Store:001"}]  } ] }'
```

Agente IoT UL
Ejecutar comandos mediante el CB

- Una vez registrado un dispositivo con comandos en el **Agente IoT**, este **se registra en el CB** como un **proveedor de contexto**
- Habilita la posibilidad de **ejecutar comandos usando directamente el CB**
- Ej.: tocar la campana

```
curl -iX PATCH \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Bell:001/attrs?type=Bell' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{
  "ring": {
      "type" : "command",
      "value" : ""
  }
}'
```

Agente IoT UL
Ejecutar comandos mediante el CB (II)

- Ej.: abrir la puerta

```
curl -iX PATCH \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Door:001/attrs?type=Door' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "open": {
          "type" : "command",
          "value" : ""  } }'
```

- Ej.: encender la lámpara

```
curl -iX PATCH \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Lamp:001/attrs?type=Lamp' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "on": {
          "type" : "command",
          "value" : "" } }'
```

Agente IoT UL
Dispositivos – Operaciones CRUD

- Mediante **POST** / **GET** / **PUT** / **DELETE** en http://<iot-agent>**:4041/iot/devices**
- Ej.: Aprovisionamiento de un dispositivo

```
curl -X POST 'http://localhost:4041/iot/devices/bell002' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /’ \
  -d '{ "devices": [
          {
            "device_id": "bell002",
            "entity_name": "urn:ngsi-ld:Bell:002",
            "entity_type": "Bell",
            "protocol": "PDI-IoTA-UltraLight",
            "transport": "HTTP",
            "endpoint": "http://context-provider:3001/iot/bell002",
            "commands": [ { "name": "ring", "type": "command" } ],
            "static_attributes": [
               { "name":"refStore", "type": "Relationship",
                 "value": "urn:ngsi-ld:Store:002"} ]
       } ] }'
```

Agente IoT UL
Dispositivos – Operaciones CRUD (II)

- Ej.: Lectura de los detalles de un dispositivo aprovisionado

```
curl -X GET 'http://localhost:4041/iot/devices/bell002' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Resp.:


```
{ "device_id": "bell002", "service": "openiot", "service_path": "/",
  "entity_name": "urn:ngsi-ld:Bell:002", "entity_type": "Bell",
  "endpoint": "http://context-provider:3001/iot/bell002",
  "polling": false,
  "transport": "HTTP",
  "attributes": [ {
     "object_id": "ring", "name": "ring", "type": "Text",
                  "expression": "${@ring}",
     "reverse": [ {
       "object_id": "ring", "type": "Text", "expression": "${@ring}" } ] } ],
  "commands": [],
  "static_attributes": [ {
     "name": "refStore", "type": "Relationship",
     "value": "urn:ngsi-ld:Store:002" } ],
  "protocol": "PDI-IoTA-UltraLight",
  "explicitAttrs": false }
```

Agente IoT UL
Dispositivos – Operaciones CRUD (III)

- Ej.: Listado de los dispositivos aprovisionados

```
curl -X GET 'http://localhost:4041/iot/devices' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Resp.:

```
{
  "count": 5,
  "devices": [
     { "device_id": "bell002", "service": "openiot", "service_path": "/",
       "entity_name": "urn:ngsi-ld:Bell:002", "entity_type": "Bell",
     …
     },
  …
  ]
}
```

Agente IoT UL
Dispositivos – Operaciones CRUD (III)

- Ej.: Actualización de un dispositivo aprovisionado

```
curl -iX PUT 'http://localhost:4041/iot/devices/bell002' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{
  "entity_type": "IoT-Device"
}'
```

- Eliminación de un dispositivo aprovisionado

```
curl -iX DELETE 'http://localhost:4041/iot/devices/bell002' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

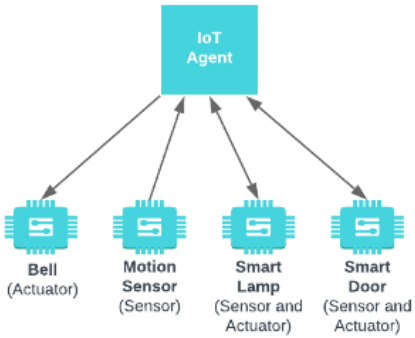
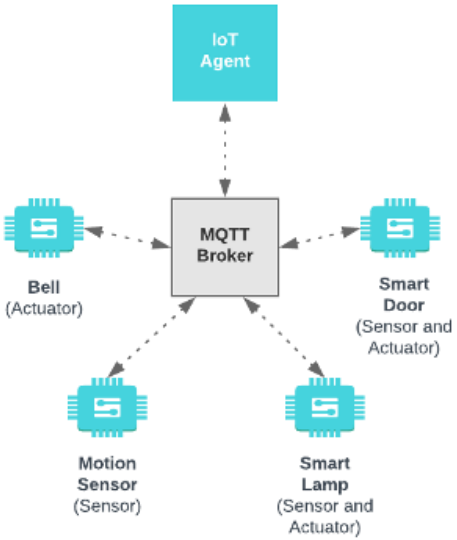
Agente IoT MQTT

Agentes IoT

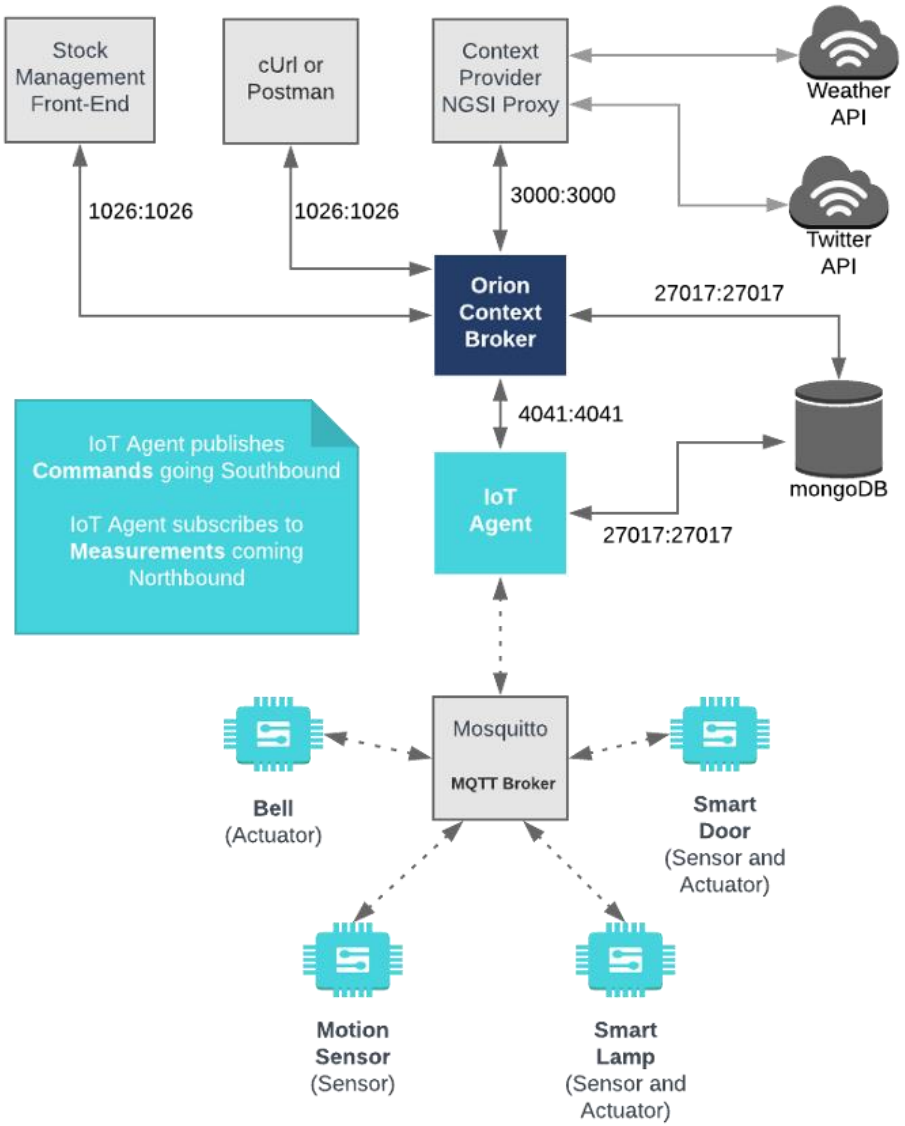
Agente IoT MQTT

- Protocolo **MQTT**
  - **Publicación – Suscripción**
  - Sobre TCP/IP
  - Tamaño de código pequeño
  - Eficiente y de bajo consumo energético
- **HTTP**
  - **Petición – Respuesta**
  - Cada dispositivo se conecta directamente al Agente IoT
- **MQTT**
  - **Requiere** punto de comunicación central adicional (**bróker MQTT**)
  - **Desacopla emisores y receptores**: un emisor envía mensajes al bróker bajo un *tópico* MQTT. Los receptores se suscriben a los tópicos para recibir los mensajes. Emisores y receptores no se conocen entre ellos
  - Altamente **escalable**

Agente IoT MQTT
HTTP Vs MQTT

HTTP	MQTT
	
Comunicación Agente IoT – Dispositivos: **directa**	**Comunicación** Agente IoT – Dispositivos: **indirecta** a través del bróker MQTT
Petición – Respuesta	**Publicación – Suscripción**
Dispositivos deben estar **siempre preparados** para recibir mensajes	Dispositivos **deciden cuándo recibir** los mensajes
Alto requerimiento de energía	**Bajo** requerimiento de energía

Agente IoT MQTT
Arquitectura del tutorial



Agente IoT MQTT
Bróker MQTT Mosquitto

- Ejecución de un suscriptor MQTT (terminal #1)

```
docker run -it --rm --name mqtt-subscriber \
  --network fiware_default efrecon/mqtt-client sub -h mosquitto -t "/#"
```

- Ejecución de un publicador MQTT (terminal #2)

```
docker run -it --rm --name mqtt-publisher \
  --network fiware_default efrecon/mqtt-client pub -h mosquitto -m
"HELLO WORLD" -t "/test"
```

  - Resultado: aparece “HELLO WORLD” en la terminal #1

- Parar ejecución del suscriptor MQTT

```
docker stop mqtt-subscriber
```

Agente IoT MQTT
Bróker MQTT Mosquitto (II)

- Log del bróker MQTT Mosquitto

```
docker logs --tail 10 mosquitto
```

  - Resultado:

```
1529661883: New client connected from 172.18.0.5 as mqttjs_8761e518 (c1, k0).
1529662472: New connection from 172.18.0.7 on port 1883.
1529662472: New client connected from 172.18.0.7 as mosqpub|1-5637527c63c1 (c1, k60).
1529662472: Client mosqpub|1-5637527c63c1 disconnected.
1529662614: New connection from 172.18.0.7 on port 1883.
1529662614: New client connected from 172.18.0.7 as mosqsub|1-64b27d675f58 (c1, k60).
1529662623: New connection from 172.18.0.8 on port 1883.
1529662623: New client connected from 172.18.0.8 as mosqpub|1-ef03e74b0270 (c1, k60).
1529662623: Client mosqpub|1-ef03e74b0270 disconnected.
1529667841: Socket error on client mosqsub|1-64b27d675f58, disconnecting.
```

  - También podemos verlo directamente en Docker Desktop

Agente IoT MQTT
Aprovis. de grupo de dispositivos MQTT

- Se utiliza la ***apikey*** como **tópico** MQTT

```
curl -iX POST 'http://localhost:4041/iot/services' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "services": [
        {
          "apikey":      "4jggokgpepnvsb2uv4s40d59ov",
          "cbroker":     "http://orion:1026",
          "entity_type": "Thing",
          "resource":    ""
        } ] }'
```

- Medidas y comandos usan **tópicos distintos**
  - **Medidas**: */<protocol>/<api-key>/<device-id>/attrs*
    - **Necesario indicar el protocolo** (UL, JSON, etc.) ya que es necesario saber qué agente MQTT (UL, JSON, etc) debe procesar la medida
  - **Comandos**: */<api-key>/<device-id>/cmd*
    - **No hace falta indicar el protocolo** (UL, JSON, etc) ya que el agente MQTT correspondiente ya se indicó cuando se aprovisionó el dispositivo

Agente IoT MQTT
Aprovisionamiento de un sensor

- Al indicar "protocol": "PDI-IoT-UltraLight" y "transport": "MQTT", el agente IoT MQTT se suscribirá al tópico */ul/<api-key>/<device-id>/attrs*

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices": [
        { "device_id":   "motion001",
          "entity_name": "urn:ngsi-ld:Motion:001",
          "entity_type": "Motion",
          "protocol":    "PDI-IoTA-UltraLight", "transport":   "MQTT",
          "timezone":    "Europe/Berlin",
          "attributes": [
             { "object_id": "c", "name": "count", "type": "Integer" } ],
     "static_attributes": [
       { "name":"refStore", "type": "Relationship",
         "value": "urn:ngsi-ld:Store:001"} ] } ] }'
```

Agente IoT MQTT
Aprovisionamiento de un sensor (II)

- Podemos **simular una medida** proveniente del sensor de movimiento mediante la publicación de un mensaje en el tópico correspondiente

```
docker run -it --rm --name mqtt-publisher --network \
  fiware_default efrecon/mqtt-client pub -h mosquitto -m "c|1" \
  -t "/ul/4jggokgpepnvsb2uv4s40d59ov/motion001/attrs"
```

- El mensaje enviado al agente IoT MQTT se puede ver en la sección *Device Monitor* de la interfaz web del tutorial

Agente IoT MQTT Aprovisionamiento de un sensor (III)

- Comprobamos que el agente IoT MQTT ha trasladado esa medida al CB

```
curl -G -X GET \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Motion:001' \
  -d 'type=Motion' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Resp.:


```
{ "id": "urn:ngsi-ld:Motion:001",
  "type": "Motion",
  "TimeInstant": {
      "type": "ISO8601",
      "value": "2018-05-25T10:51:32.00Z", "metadata": {} },
  "count": {
      "type": "Integer",
      "value": "1",
      "metadata": { "TimeInstant": {
                        "type": "ISO8601",
                        "value": "2018-05-25T10:51:32.646Z" } } } }
```

Agente IoT MQTT
Aprovisionamiento de un actuador

- Al indicar "protocol": "PDI-IoT-UltraLight" y "transport": "MQTT", el agente IoT MQTT se suscribirá al tópico */<api-key>/<device-id>/cmd*

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{
  "devices": [
    {
      "device_id": "bell001",
      "entity_name": "urn:ngsi-ld:Bell:001", "entity_type": "Bell",
      "protocol": "PDI-IoTA-UltraLight", "transport": "MQTT",
      "apikey": "4jggokgpepnvsb2uv4s40d59ov",
      "commands": [ { "name": "ring", "type": "command" } ],
       "static_attributes": [
         { "name":"refStore", "type": "Relationship",
           "value": "urn:ngsi-ld:Store:001"} ] } ] }'
```

Agente IoT MQTT Aprovisionamiento de un actuador (II)

- Podemos **simular el envío de un comando** mediante la petición NGSI al agente IoT MQTT tal y como hará el CB

```
curl -iX POST http://localhost:4041/v2/op/update \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{
    "actionType": "update",
    "entities": [ {
            "type": "Bell",
            "id": "urn:ngsi-ld:Bell:001",
            "ring" : { "type": "command", "value": "" } } ] }'
```

- El mensaje enviado al agente IoT MQTT se puede ver en la sección *Device Monitor* de la interfaz web del tutorial

Agente IoT MQTT
Aprovisionamiento de un actuador (III)

- Comprobamos resultado ejecución del comando en CB

```
curl -G -X GET 'http://localhost:1026/v2/entities/urn:ngsi-ld:Bell:001' \
  -d 'type=Bell' \
  -d 'options=keyValues' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Resp.:


```
{
    "id": "urn:ngsi-ld:Bell:001",
    "type": "Bell",
    "TimeInstant": "2018-05-25T20:06:28.00Z",
    "refStore": "urn:ngsi-ld:Store:001",
    "ring_info": "OK",
    "ring_status": "OK",
    "ring": ""
}
```

Agente IoT MQTT
Aprovis. sensor & actuador *(Smart Door)*

- El agente IoT MQTT se suscribirá a los tópicos
  - */ul/<api-key>/<device-id>/**attrs***
  - */<api-key>/<device-id>/**cmd***

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices": [
    { "device_id": "door001",
      "entity_name": "urn:ngsi-ld:Door:001", "entity_type": "Door",
      "protocol": "PDI-IoTA-UltraLight", "transport": "MQTT",
      "apikey": "4jggokgpepnvsb2uv4s40d59ov",
      "commands": [ {"name": "unlock","type": "command"},
                    {"name": "open","type": "command"},
                    {"name": "close","type": "command"},
                    {"name": "lock","type": "command"} ],
      "attributes": [ {"object_id": "s", "name": "state",
                       "type":"Text"} ],
      "static_attributes": [ {"name":"refStore",
         "type": "Relationship", "value": "urn:ngsi-ld:Store:001"}]}]}'
```

Agente IoT MQTT
Aprovis. sensor & actuador *(Smart Lamp)*

- El agente IoT se suscribirá a los tópicos
  - */ul/<api-key>/<device-id>/**attrs***
  - */<api-key>/<device-id>/**cmd***

```
curl -iX POST 'http://localhost:4041/iot/devices' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "devices": [
    { "device_id": "lamp001",
      "entity_name": "urn:ngsi-ld:Lamp:001", "entity_type": "Lamp",
      "protocol": "PDI-IoTA-UltraLight", "transport": "MQTT",
      "apikey": "4jggokgpepnvsb2uv4s40d59ov",
      "commands": [ {"name": "on","type": "command"},
                    {"name": "off","type": "command"} ],
      "attributes": [
        {"object_id": "s", "name": "state", "type":"Text"},
        {"object_id": "l", "name": "luminosity", "type":"Integer"} ],
      "static_attributes": [ {"name":"refStore", "type": "Relationship",
          "value": "urn:ngsi-ld:Store:001"} ] } ] }'
```

Agente IoT MQTT
Tareas finales

- Listado de los dispositivos aprovisionados

```
curl -X GET 'http://localhost:4041/iot/devices' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /'
```

- Ejecución de comandos mediante el CB
  - La interfaz CB <-> Agente IoT siempre es NGSI: el proceso es exactamente el mismo para cualquier tipo de agente IoT
  - Reutilizamos las peticiones NGSI que usamos para el agente IoT HTTP UltraLight. Ej.:

```
curl -iX PATCH \
  'http://localhost:1026/v2/entities/urn:ngsi-ld:Door:001/attrs' \
  -H 'Content-Type: application/json' \
  -H 'fiware-service: openiot' -H 'fiware-servicepath: /' \
  -d '{ "open": { "type" : "command", "value" : "" } }'
```